CSTAM 3.0 – TuniBot Challenge

TuniBot

Autonomous Service Robot for Indoor Delivery

Technical Report – Phase 3: UI/UX and Documentation

Team ConnectENIT

Wassim Toukebri Maram Hammami Ridha Charrad

IEEE ENIT Computer Society Student Branch Chapter



November 1, 2026

Repository: https://github.com/maramhammami-svg/tunibot-phase3

Contents

Abstract **2**

1 Team Overview and Contributions **2**

2 Problem Statement and Industrial Context **2**

3 Proposed Solution **3**
3.1 Phase 3 requirements coverage 3

4 System Design **4**
4.1 Architecture 4
4.2 The `cstam_web` package 4
4.3 ROS 2 interface 6
4.4 Order tracking without changing the task manager 6
4.5 Voice ordering pipeline (bonus feature) 6

5 Mockups **7**
5.1 Design direction 7
5.2 Client page 7
5.3 Admin control room 9

6 Technologies, Frameworks and Tools **10**

7 Development Milestones **11**

8 Testing Results **11**
8.1 Automated tests 11
8.2 Validation in Gazebo 12
8.3 Usability checks 12

9 Future Improvements **12**

References **13**

Abstract

We built a complete ROS 2 software stack that runs an autonomous waiter robot in a simulated restaurant, from the guest's order to the robot's return to its charger, with no human in the loop. The stack runs on ROS 2 Jazzy, Gazebo Harmonic and Nav2. It covers mapping and localization (SLAM, AMCL), autonomous navigation, a delivery task queue, auto-docking, dynamic obstacle avoidance, recovery in narrow aisles and a physical battery model with a return-to-dock policy.

Phase 3 adds the user layer. A web *Client* page lets guests choose their table on the real floor plan, order from an illustrated menu book by tapping or by voice (French, English and Tunisian Darija), place several orders at once and follow each delivery live with an arrival estimate. An *Admin* control room shows the robot on a live floor map with its mission, battery, order queue, system health and event log, and lets staff send, recall or cancel. The interface is a separate ROS 2 package, `cstam_web`, that only uses the topics the robot already publishes, so no Phase 1 or Phase 2 code was changed. A web order was delivered end to end in Gazebo, and 51 automated tests (unit, API and browser end-to-end) pass on the interface.

1 Team Overview and Contributions

We are **ConnectENIT**, a team of three students from the IEEE ENIT Computer Society Student Branch Chapter. All three of us worked on every phase of the challenge: we designed, coded, tested and reviewed together. To move fast, each member led one area in each phase while the others reviewed and tested it, as summarised in Table 1.

Member	Phase 1: Foundations	Phase 2: Integration	Phase 3: UI/UX & docs
Wassim Toukebri	Nav2 navigation set-up, waypoints to tables	Dynamic obstacle avoidance, costmap tuning for narrow doorways, recovery behaviours	FastAPI server and ROS 2 bridge, order tracking, Gazebo validation
Maram Hammami	Delivery task queue manager and auto-docking	Battery model and return-to-dock policy, Dar Tunibot world	Client page UX and visual design, voice ordering (French, English, Darija), technical report
Ridha Charrad	Gazebo world and robot model, SLAM mapping, AMCL localization	Integration tests, pedestrian scenarios	Admin control room, automated tests, demo video

Table 1: Each member's lead area per phase. All members contributed to all three phases.

2 Problem Statement and Industrial Context

Restaurant service still depends on staff walking every plate and drink from the kitchen to the table. That repetitive work takes time away from guests, and it scales badly at peak hours.

The CSTAM 3.0 challenge, set with our industrial partner TuniBot, a Tunisian robotics and automation company, asks for a software-only answer. The robot must:

- understand and navigate an indoor environment it has mapped itself;
- receive food and beverage requests from users;
- drive to predefined tables to deliver them, safely around people and furniture;
- return to its dock to charge when idle or low on battery;

- give guests and staff an interface to order and to supervise the robot (Phase 3).

Two constraints shaped the design. Everything runs in simulation with industry-standard open-source tools (ROS 2, Gazebo, Nav2), so the stack can move to a real base later. And the result must work in a realistic venue: we tested in a large lounge (24 tables) and in *Dar Tunibot*, a traditional Tunisian restaurant with 1.20 m arched doorways and narrow aisles.

For Phase 3 specifically, the problem is trust and ease of use. A guest who orders from a robot needs to know the order was received, where the robot is and when it will arrive. Staff need to see at a glance whether the robot is healthy, how much battery it has left and what is waiting, and to take over when something goes wrong.

3 Proposed Solution

Our solution is a layered ROS 2 system: a simulated restaurant and robot at the bottom, the robot's core logic in the middle and a web interface on top. Each layer talks to the next only through ROS 2 topics, so each can be tested and replaced on its own.

Layer	**What it does**	**Phase**
Simulation	Gazebo Harmonic worlds (Restaurant Lounge, 24 tables; Dar Tunibot, 10 tables), BellaBot differential-drive model with 2D LiDAR and wheel odometry, a walking pedestrian	1, 2
Mapping and localization	SLAM to build the map, AMCL to localize, auto-initialized at the dock	1
Navigation	Nav2 planner, controller, behaviour tree and recoveries; costmaps tuned for 1.20 m doorways (robot radius 0.28 m, inflation 0.42 m)	1, 2
Delivery task manager	Queue of delivery requests, dispatch to tables, completion, cancel, recovery that keeps the order in the queue	1
Auto-docking	Return to dock when the queue is empty, docking sequence, rest	1
Dynamic obstacles	Pedestrian controller (patrol, cross, block); 10 Hz local costmap so Nav2 re-plans around people	2
Battery	24 V 20 Ah Li-ion model with voltage sag and temperature; return to dock at 25 %, emergency dock at 15 %, recharge to 80 %	2
Web interface	Client ordering page with voice orders, Admin control room, FastAPI bridge to ROS 2	3

Table 2: Layers of the TuniBot software stack.

3.1 Phase 3 requirements coverage

The specification asks for an intuitive interface (web or mobile) to request deliveries, and an Admin view to monitor the robot's status and battery. Table 3 maps each requirement to what we built.

Requirement	Delivered
Easy delivery requests	Client page: tap your table on the floor plan (or open `/?table=5` from a table QR code), add dishes from the menu book, one Order button; or say the order out loud
Web and mobile	Responsive page served on the restaurant Wi-Fi: one column on phones, two columns on wide screens; phones open `http://<PC-IP>:8000/`
Customer feedback	Live tracking per order: received, waiting, on the way (with arrival estimate), at your table, delivered; cancel while still queued
Several orders	A guest can place a new order while earlier ones are still on their way and switch between them
Admin: robot status	Mission card, robot position on a live floor map with the Nav2 path, system health, event log
Admin: battery	Percentage ring and gauge, voltage, current, power, temperature, runtime, 5-minute history with the 25 % and 15 % thresholds
Admin: control	Cancel any order, send the robot to a table or the kitchen, return to dock, demo buttons for battery level and pedestrian
Bonus: voice orders	Microphone button, French / English / Tunisian Darija, cart filled and read back for confirmation

Table 3: Phase 3 requirements and how they are met.

4 System Design

4.1 Architecture

The web interface sits on top of the robot stack and reaches it only through the ROS 2 topics the Phase 1 and Phase 2 nodes already use (Figure 1). The robot keeps working if the web server stops, and the web server never runs its own copy of the queue or the battery.

4.2 The `cstam_web` package

- `ros_bridge.py`: one ROS 2 node that subscribes to the robot topics and publishes the web commands.
- `store.py`: a thread-safe telemetry store that mirrors what the robot publishes and derives each web order's status.
- `server.py`: FastAPI REST endpoints, plus a WebSocket that pushes a full snapshot twice a second.
- `voice.py`: the voice-order parser (plain Python, no ROS).
- `static/`: Client and Admin pages in plain HTML, CSS and JavaScript, with no build step. Dish illustrations and icons are drawn as inline SVG (`dishes.js`, `icons.js`), and the floor map is drawn on an HTML canvas from the world file (`map.js`).
- `mock.py`: a simulated robot that reuses the real task queue and battery classes, for demos without Gazebo (`--mock`).

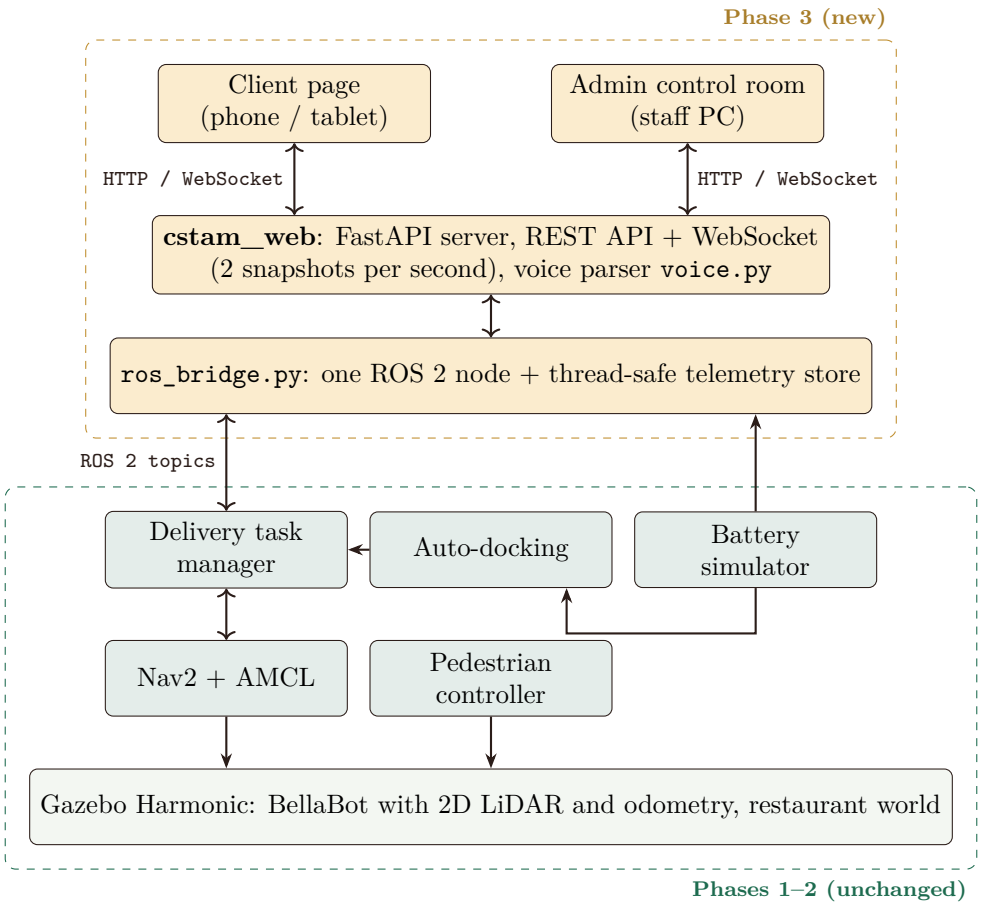


Figure 1: System architecture. The Phase 3 package only talks to the robot through existing ROS 2 topics.

4.3 ROS 2 interface

Topic	Direction	Content
`/delivery_request`	web → robot	`{"target": "Table 4", "item": "2x Thé à la menthe"}`
`/delivery_task_action`	web → robot	cancel a task by id
`/dock_command`	web → robot	`DOCK_NOW:ADMIN` or `STOP`
`/delivery_queue_status`	robot → web	current task, pending tasks, completed count, battery lock (JSON)
`/robot_system_state`	robot → web	idle, navigating, at_table, recovering, docking, docked
`/battery_status`	robot → web	percentage, voltage, current, power, temperature, flags (JSON, 1 Hz)
`/amcl_pose`, `/plan`	robot → web	robot pose and Nav2 global path for the map
`/dynamic_obstacle/status`	robot → web	pedestrian position and mode
`/delivery_waypoints`	robot → web	world name and table coordinates (latched)

Table 4: ROS 2 topics used by the web interface.

4.4 Order tracking without changing the task manager

The Phase 1 task manager knows nothing about web orders. The bridge links an order to its task by watching the queue status: the first new task id for the same table with the same item text is that order. From then on its status follows the task: queued, en route, at the table. When it leaves the queue, a rise in the completed count means delivered; otherwise it is reported as failed or cancelled. An order the task manager does not pick up within 8 s is shown as not accepted. Because each order is tracked on its own, a guest can follow several orders at once; they are kept in the browser's local storage so tracking resumes after a reload.

4.5 Voice ordering pipeline (bonus feature)

A guest taps the microphone and says the order in one sentence, for example *"deux thés à la menthe et un brik pour la table 4"* or, in Darija, *"zouz kahwa w brik, tawla tletha"*. Nothing is sent to the robot without the guest's confirmation tap.

1. The browser turns speech into text (Web Speech API) in the language chosen in the voice panel: French (default), English or Darija (`ar-TN`). Listening stops after 2.5 s of silence.
2. Up to five recognition guesses are sent to `POST /api/voice/parse`.
3. The parser reads each guess against the current world's menu and tables and keeps the best fit: most dishes recognised, a table found, fewest unknown words.
4. The page fills the cart, selects the table, opens the menu page of the first dish and reads the order back aloud.

The parser understands quantities in French, English and Darija ("deux", "three", "zouz", "tletha"), table phrases ("table numéro 10", "tawla tletha"), dish aliases ("kahwa", "atay", "kouksi") and the misspellings speech-to-text produces for Tunisian words ("brique" for brik, "bamba louni" for bambalouni). Chrome returns Darija in Arabic script, so known words are mapped directly and the rest is transliterated to Latin letters before matching. The parser uses accent folding, plural removal, a phonetic key and fuzzy matching. A dish that is not on the menu is reported, never guessed.

5 Mockups

The screens below are the working interface, captured from the running system in the Dar Tunibot world. They are the final UI, not drawings.

5.1 Design direction

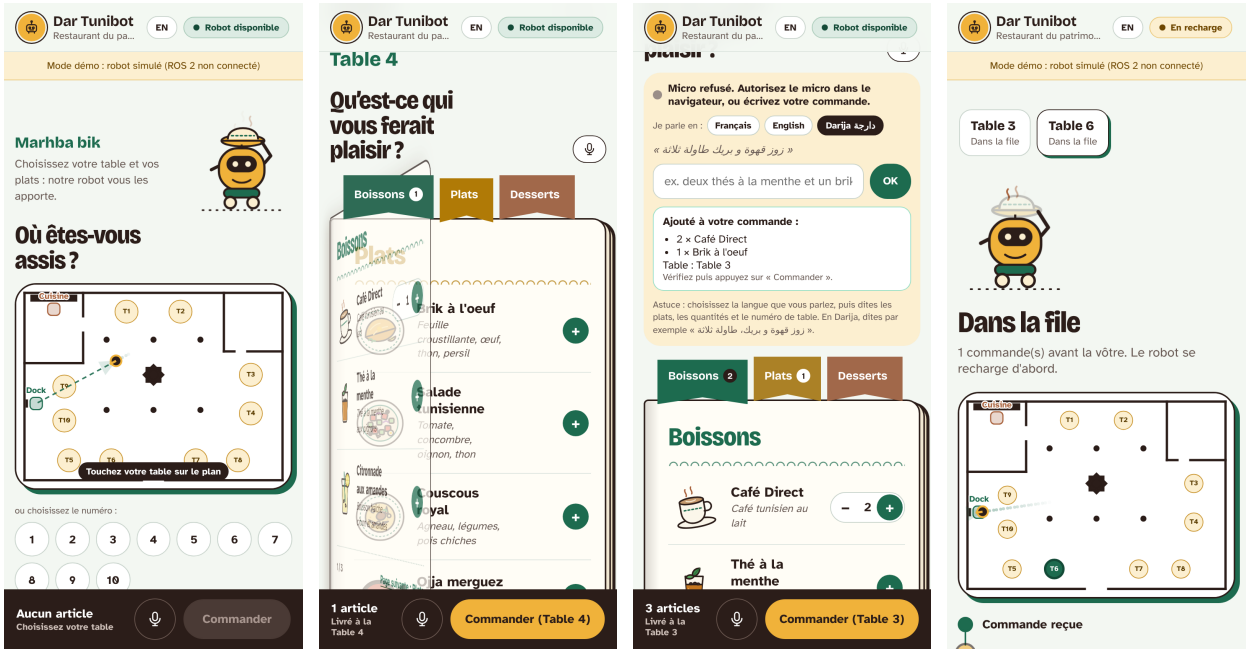
We wanted the interface to feel like it belongs in a Tunisian restaurant rather than a generic dashboard. The visual identity borrows from Nabeul pottery: copper green, ochre and manganese brown on a pale slip background. Headings use Bricolage Grotesque; body text uses Atkinson Hyperlegible, a typeface designed for readability. Each dish has its own flat illustration (Figure 2), and the menu behaves like a real menu book with page turns. Motion is used only where it carries meaning (a dish flying to the tray, the robot driving on the map) and is turned off for users who ask their system for reduced motion.



Figure 2: Dish illustrations drawn for the Tunisian menu.

5.2 Client page

The guest orders in three taps: table, dishes, Order. Everything else on the page answers the guest's next question: is the robot free, where is my order, when will it arrive.



(a) Pick the table on the real floor plan.

(b) Menu book, one page per category.

(c) Voice order in Darija.

(d) Several orders tracked at once.

Figure 3: Client page on a phone.

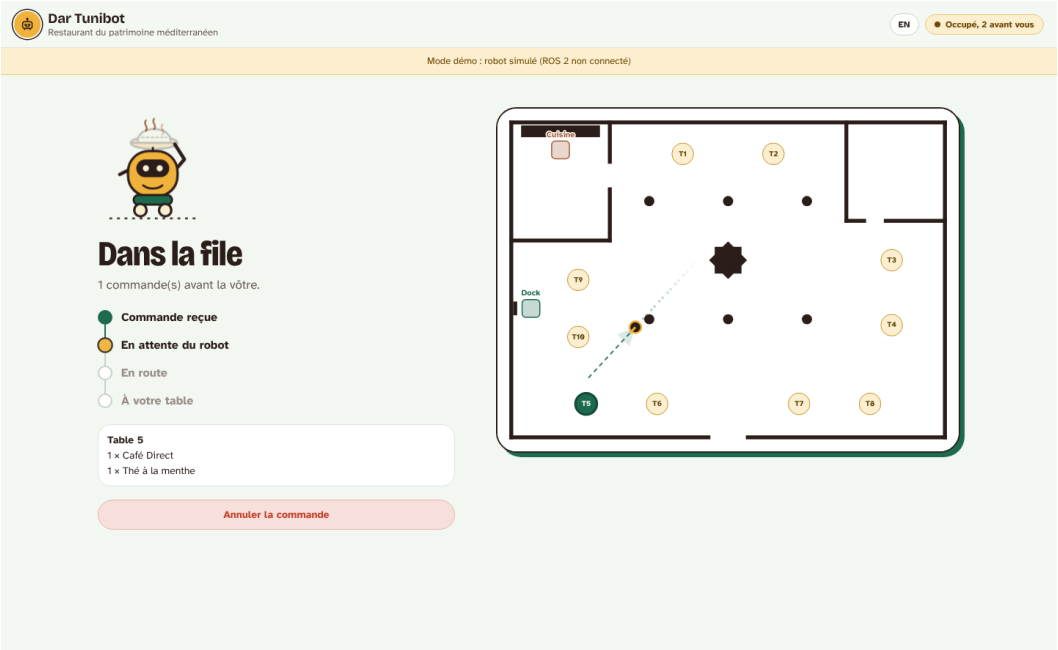


Figure 4: Client page on a wide screen: the floor plan stays visible beside the order and the live tracking.

Edge cases handled.

- Robot offline (no battery heartbeat for 3 s): red banner, ordering disabled.
- Battery lock (robot recharging to 80 %): order accepted; the guest is told it leaves after charging.
- Order not picked up by the task manager within 8 s: shown as “not accepted, please ask the staff”.

- Navigation failure: “a waiter will bring your order” instead of a silent stop.
- Page reloaded or phone locked: orders are kept and tracking resumes.
- Lost Wi-Fi: the live connection reconnects on its own.

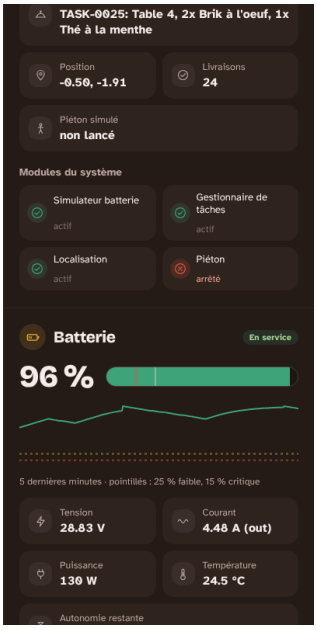
Language. French by default, English with one tap. The choice is remembered, and dish descriptions and categories are translated. Voice recognition has its own language choice, because recognition must follow the language the guest speaks, not the page language.

5.3 Admin control room

The Admin view at `/admin` shows, on one screen, everything staff need to trust the robot: where it is, what it is doing, how much battery it has left and what is waiting. The live floor map takes most of the screen. A mission card floats over it, and the side column keeps every panel open (orders, robot, battery, log, demo mode) so nothing is hidden behind tabs.



(a) Live map with the Nav2 path, mission card and order tickets.



(b) Robot and battery panels.

Figure 5: Admin control room.

Panel	Shows	Actions
Floor map	Walls, tables, dock and kitchen from the world file; robot pose from AMCL, its trail, the Nav2 path, the pedestrian, pending orders per table	Tap a table to send the robot there
Mission card	Current mission, state, battery ring, deliveries completed	–
Orders	Current and pending orders as kitchen tickets	Cancel any order
Robot	Current task, position, deliveries, pedestrian, health of each module	Return to dock, cancel the return, send to kitchen or a table
Battery	Percentage, gauge, voltage, current, power, temperature, runtime, 5-minute history	–
Log	Last events: orders, state changes, battery warnings, voice orders	–
Demo mode	–	Set battery to 100 %, 24 % or 14 %; pedestrian patrol, cross, block or absent

Table 5: Admin panels.

6 Technologies, Frameworks and Tools

Area	Technology	Use
Middleware	ROS 2 Jazzy	Nodes, topics, launch files, colcon build
Simulation	Gazebo Harmonic	Restaurant worlds, BellaBot model, LiDAR, pedestrian
Navigation	Nav2, AMCL, SLAM Toolbox	Path planning, control, recoveries, localization, mapping
Visualisation	RViz 2	Map, costmaps and markers during development
Robot logic	Python 3.12 (rclpy)	Task manager, docking, battery, obstacle controller
Web server	FastAPI, Uvicorn, Pydantic	REST API, WebSocket, request validation
Front end	HTML5, CSS3, JavaScript	Client and Admin pages (no framework), canvas floor map, SVG illustrations
Speech	Web Speech API	Speech recognition and read-back in the browser
Testing	pytest, Playwright (Chromium)	Unit, API and browser end-to-end tests
Tools	Git, GitHub, VS Code	Version control, pull requests, development on Ubuntu 24.04
Design	Google Fonts	Typography: Bricolage Grotesque, Atkinson Hyperlegible

Table 6: Technologies used.

7 Development Milestones

Milestone	Delivered
Phase 1 – Foundations	Gazebo restaurant world and BellaBot model; SLAM map; AMCL localization; Nav2 navigation to tables; delivery task queue manager; auto-docking; Mission Control simulator UI
Phase 2 – Integration and testing	Dynamic pedestrian (patrol, cross, block) with a 10 Hz local costmap; Dar Tunibot world with 1.20 m doorways and tuned inflation; multi-stage recovery that keeps the order queued; physical battery model with 25 % / 15 % / 80 % return-to-dock policy
Phase 3, step 1 – Web interface	New `cstam_web` package: Client ordering page, Admin monitoring page, ROS 2 bridge, mock mode; web order delivered end to end in Gazebo
Phase 3, step 2 – UX and voice	Voice orders (French, English, Darija), arrival estimate, order review, French/English pages, edge cases, automated tests
Phase 3, step 3 – Design	Nabeul pottery identity, dish illustrations, menu book, table picked on the floor plan, wide-screen layout, several orders at once, Admin control room
Phase 3, step 4 – Documentation	This report and the demonstration video

Table 7: Development milestones.

8 Testing Results

8.1 Automated tests

The `cstam_web` package has 51 automated tests, all passing. They run without Gazebo, against the mock robot that reuses the real task queue and battery classes.

Suite	Tests	What it checks
Store and API (`test_web.py`)	16	Order lifecycle follows the task manager; order matched by table and item; cancelled, rejected and failed orders; offline until battery heartbeat; battery lock; validation; admin commands; WebSocket snapshot
Voice parser (`test_voice.py`)	28	French, English, Darija (Latin and Arabic script) sentences; sound-alike spellings; unknown dish reported; table must exist; quantity cap; best alternative kept
Browser end-to-end (`test_e2e_browser.py`)	7	Voice order delivered end to end; microphone flow; table picked on the floor plan; menu book; language switch; admin map click sends the robot; several orders at once
Total	**51**	

Table 8: Automated test suites (parametrised cases counted individually).

Run them with:

```
cd ros2_ws/src/cstam_web
PYTHONPATH=.:../cstam_core python3 -m pytest -q test
```

8.2 Validation in Gazebo

With the full stack running (Gazebo, Nav2, task manager, battery), we ordered from the Client page and watched the robot deliver: the web order became task `TASK-0003` for Table 4, the robot drove there along the Nav2 path shown on the Admin map, the Client page moved through *on the way*, *at your table* and *delivered*, and the robot returned to its dock. We also used the Admin demo buttons to check that a 24 % battery sends the robot home after its queue, that 14 % triggers an emergency return and that the pedestrian modes make Nav2 re-plan around the person.

8.3 Usability checks

- Phone (390 px), tablet and wide-screen (1440 px) layouts checked with screenshots after each change.
- Voice tested with real speech: this showed that the recognition language must be chosen separately from the page language.
- Text contrast follows the palette's dark-on-light and light-on-dark pairs; motion is disabled when the system asks for reduced motion.

9 Future Improvements

- **Real robot:** run the same stack on a physical base; the web layer needs no change since it only uses ROS 2 topics.
- **Fleet:** several robots on one Admin map, with the task manager assigning orders to the nearest free robot.

- **Kitchen screen:** a third page for the kitchen to mark an order ready before the robot leaves.
- **Payment and accounts:** pay at the table and keep order history for loyalty.
- **Offline speech:** an on-device speech model (e.g. Whisper) so voice ordering works without Chrome's cloud service and with better Darija accuracy.
- **Security:** HTTPS and staff login for the Admin page before use in a real venue.
- **Analytics:** delivery times, battery cycles and busiest tables over a day.

References

[1] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, 2022.

[2] S. Macenski, F. Martín, R. White, J. Ginés Clavero, "The Marathon 2: A Navigation System," *IEEE/RSJ IROS*, 2020.

[3] S. Macenski, I. Jambrecic, "SLAM Toolbox: SLAM for the dynamic world," *Journal of Open Source Software*, vol. 6, no. 61, 2021.

[4] S. Thrun, W. Burgard, D. Fox, *Probabilistic Robotics*. MIT Press, 2005.

[5] Open Robotics, "ROS 2 Jazzy documentation," https://docs.ros.org/en/jazzy/.

[6] Open Robotics, "Gazebo Harmonic documentation," https://gazebosim.org/docs/harmonic.

[7] Nav2 project, "Nav2 documentation," https://docs.nav2.org/.

[8] S. Ramírez, "FastAPI," https://fastapi.tiangolo.com/.

[9] W3C Community Group, "Web Speech API," https://webaudio.github.io/web-speech-api/.

[10] Microsoft, "Playwright for Python," https://playwright.dev/python/.

[11] W3C, "Web Content Accessibility Guidelines (WCAG) 2.1," 2018.

[12] Braille Institute, "Atkinson Hyperlegible typeface," https://www.brailleinstitute.org/freefont/.